How Do We Interact with AI?

Prompt Engineering

Keyvaun Herring, Lorena Salazar, Kyla Thorpe

Pre-Survey

What is TAP?

Technology Ambassadors Program

TAP was designed as a program to encourage students to pick up a minor or major in IT. Combining service learning with hands on project development, the program provides:

- Resume building opportunities
- Community outreach
- Project of choice
- ITEC 4000 Elective credits

Our Members

Keyvaun Herring

Lorena Salazar

Kyla Thorpe

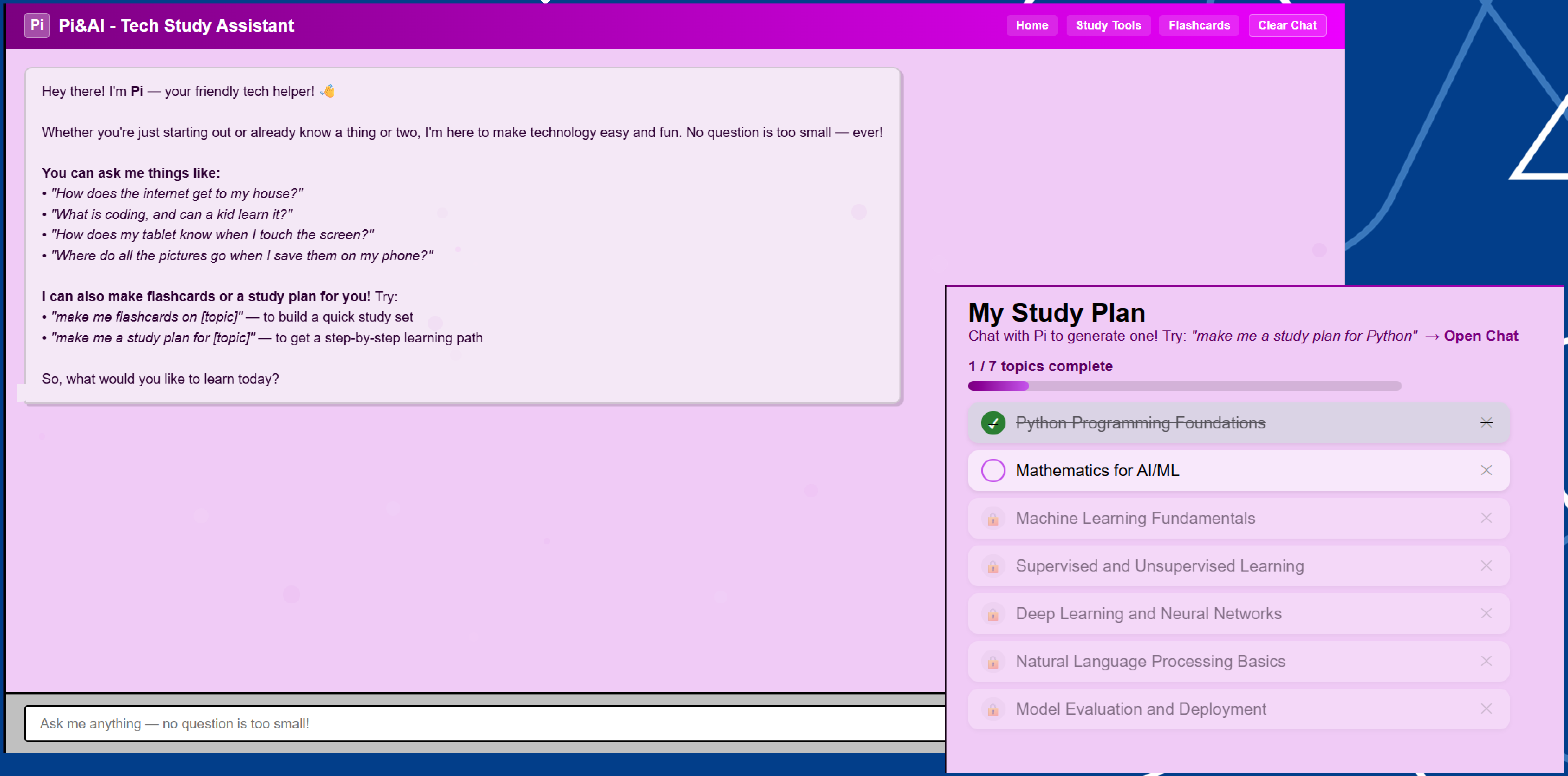
Goals

The goal for this workshop is to inspire people to build their own generative AI projects by highlighting how learning and prompt engineering impact the output the model gives.

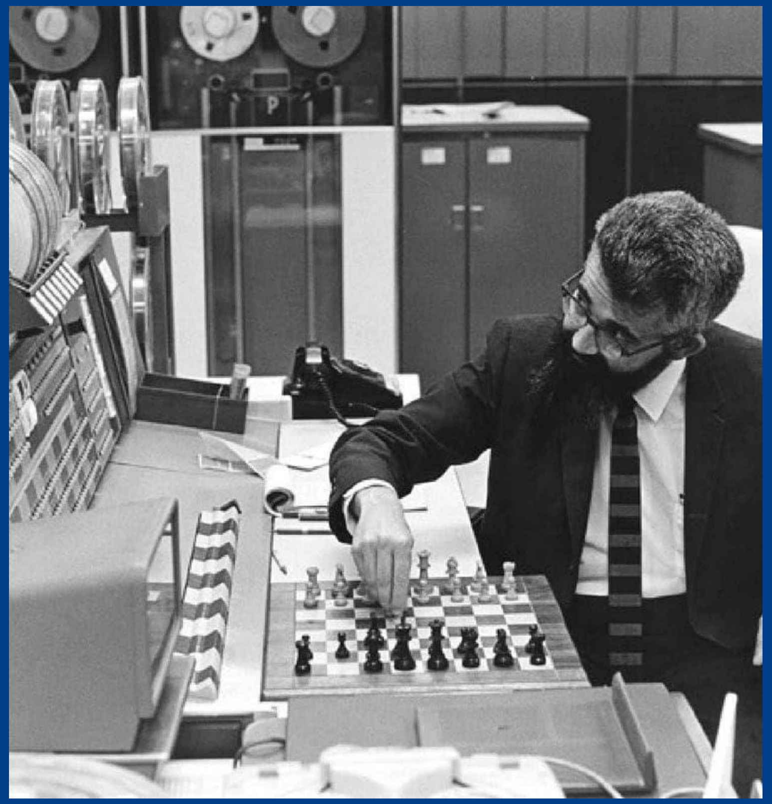
We will learn about:

- History of AI
- Subsets of AI Learning
- Prompt Engineering
    - And how you can do it too!

Our Project: PI&AI

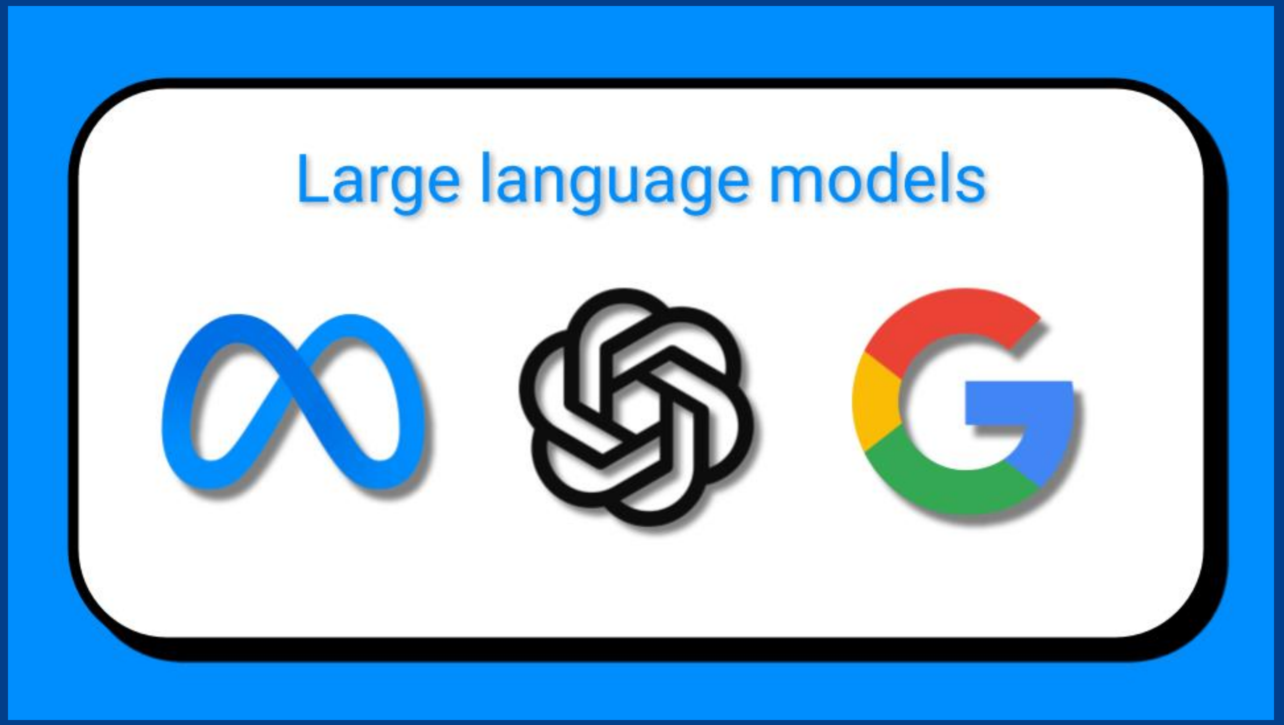


AI Learning From Then to Now



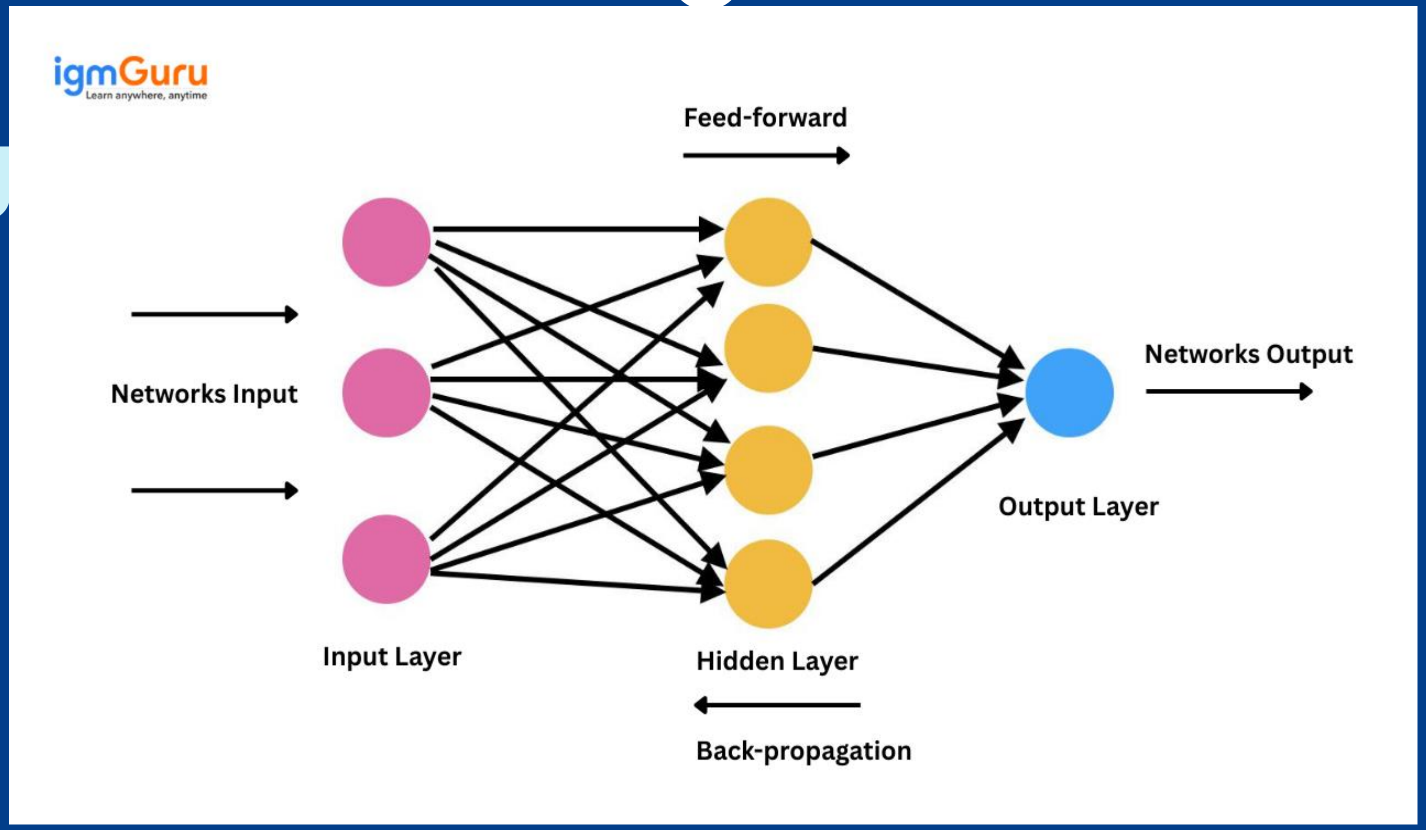
1980's - Revival of Neural Networks

Innovations like backpropagation, connectionism, and the Boltzmann machine amidst AI slows.



1950's - the Beginning

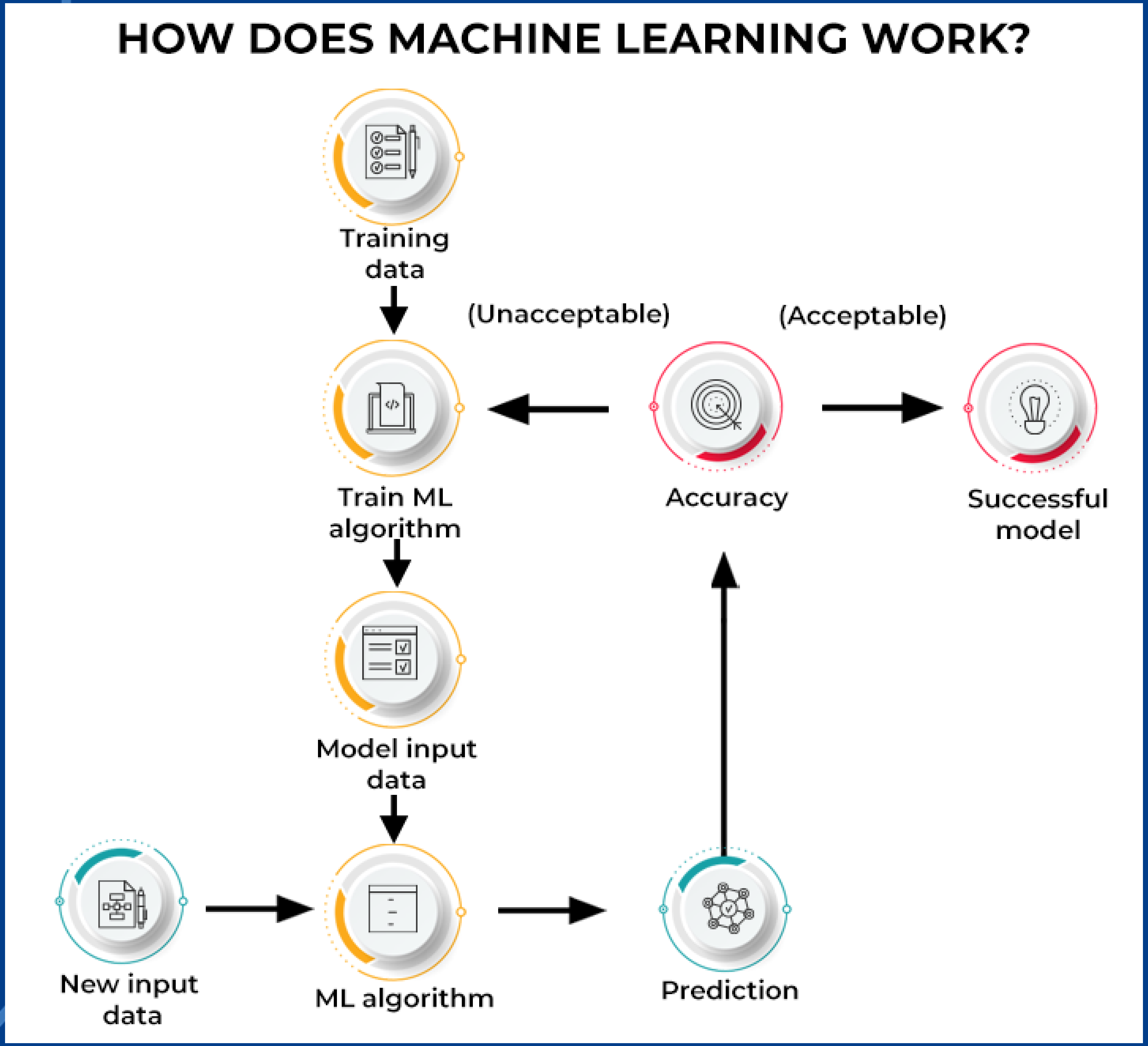
The first AI were built to solve math theorems or play chess against humans (Turochamp).



2017 - Large Language Models

The transformer architecture led to the release of GPT-3 in 2020, the predecessor of modern ChatGPT.

How Does AI Learn?



AI use algorithms applied to an input to create an output for the user, based on the data it is trained on.

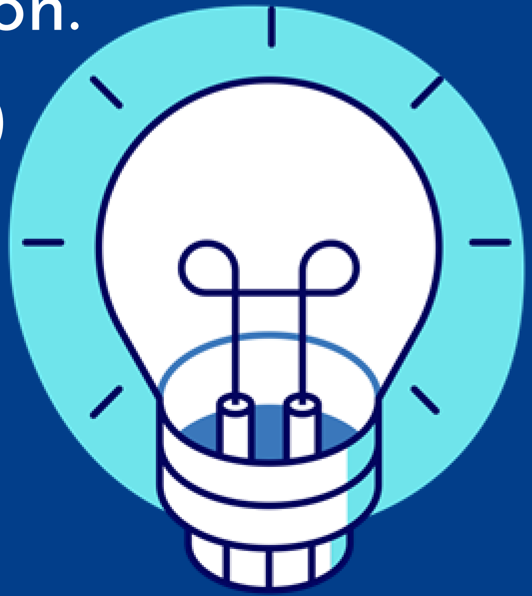
The models are given massive data sets, where these algorithms search for patterns and adjust parameters based on output results. This is called **Machine Learning.**

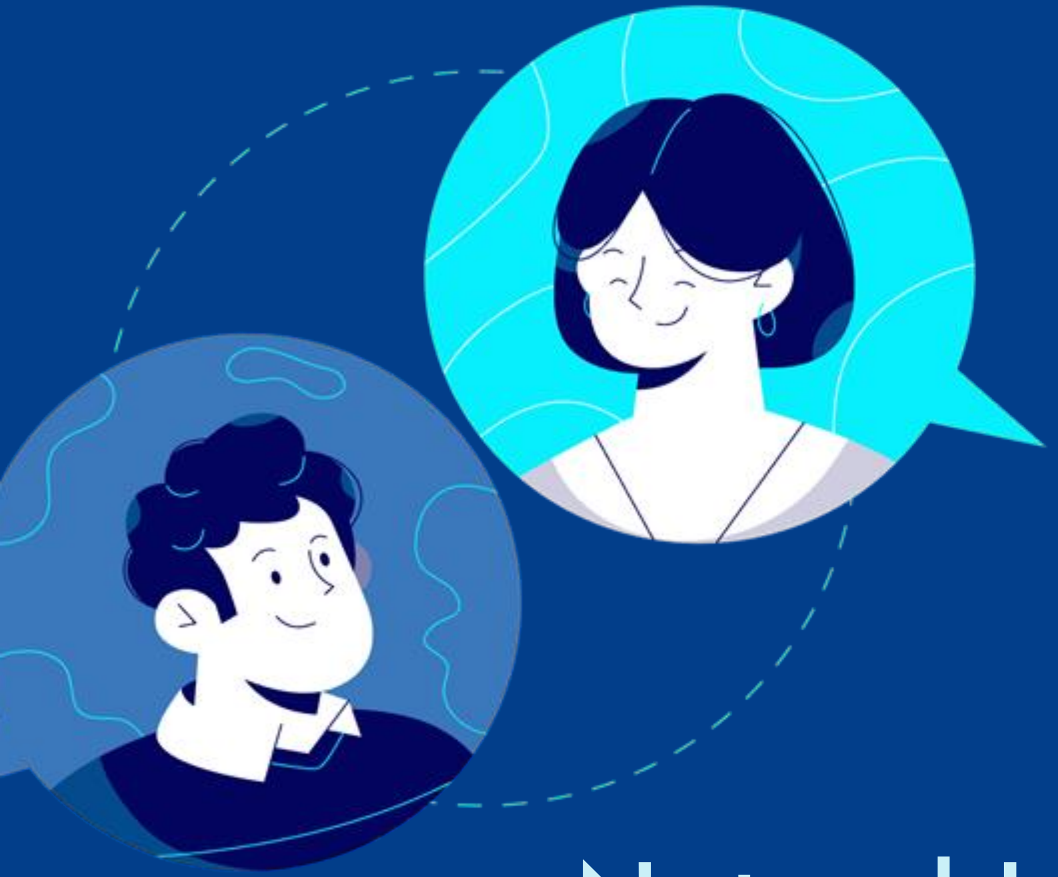
AI

Machine Learning

AI uses algorithms to improve output through patterns and prediction.

(Focus on numbers, text, images)

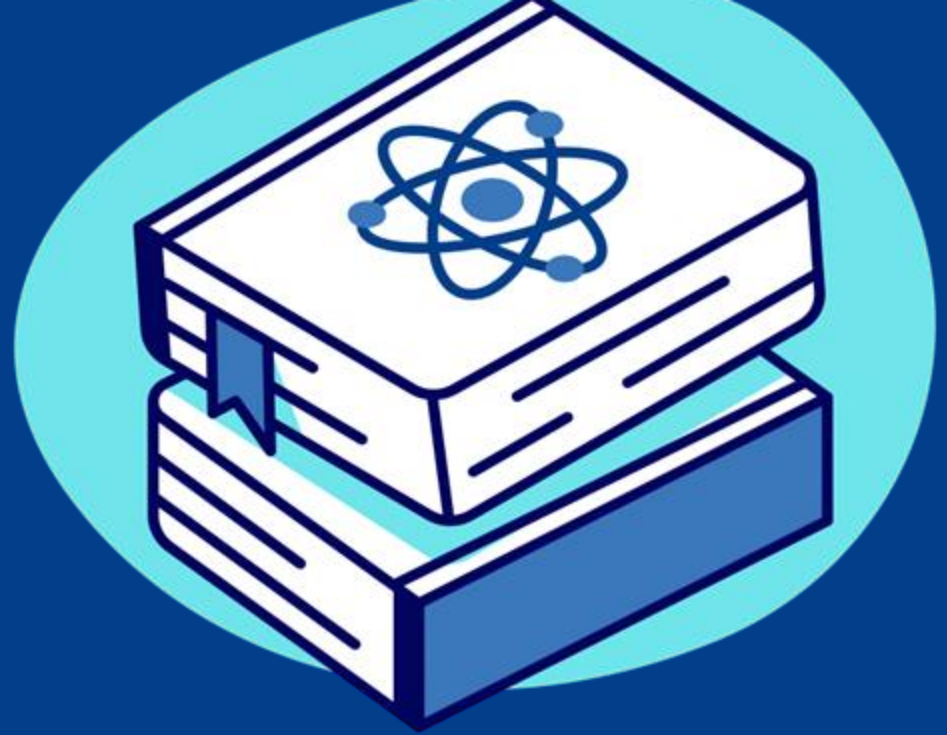




Natural Language Processing

Computers learn how to understand human language to create real time conversations

(Trained on human language)

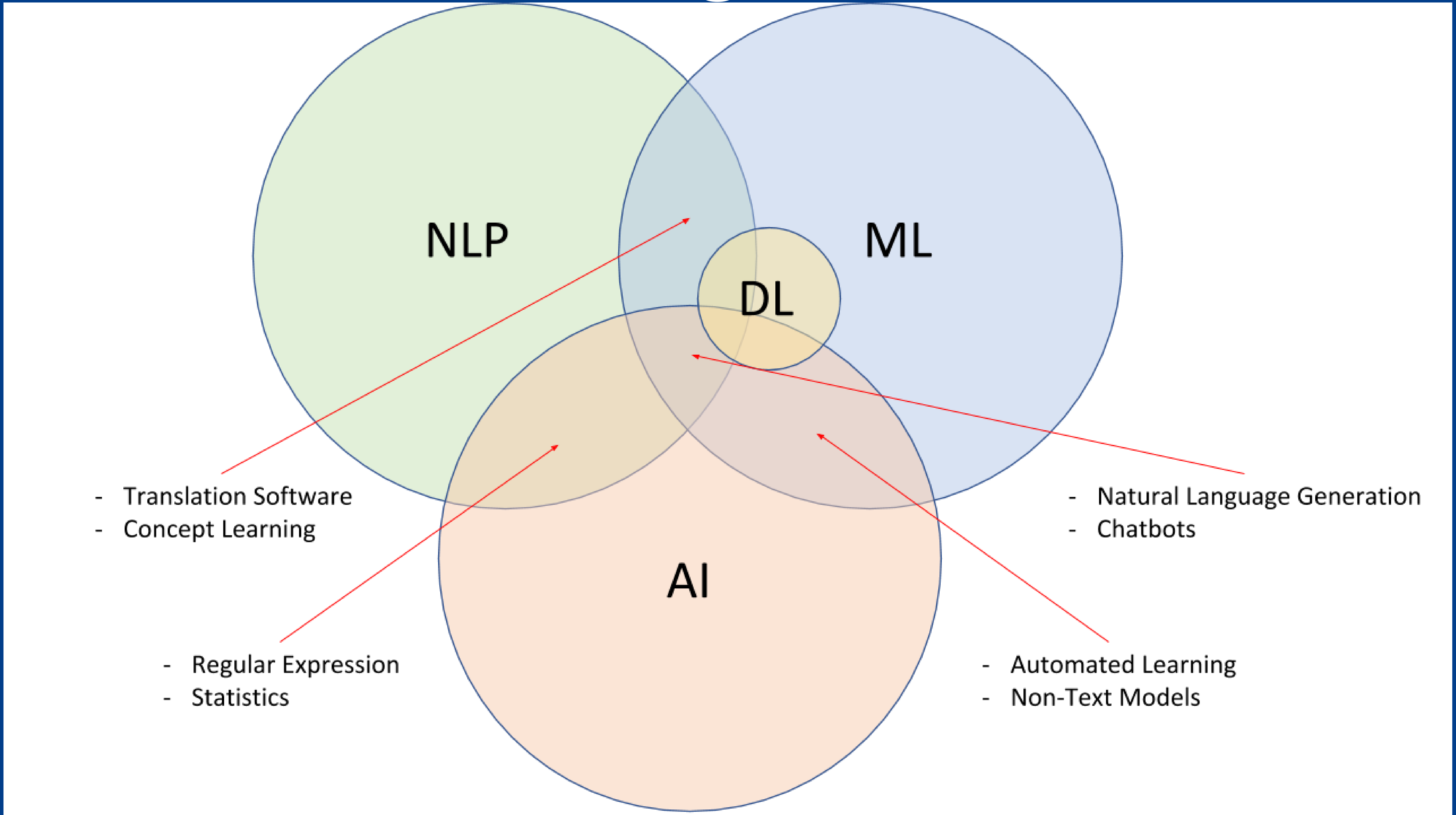


Deep Learning

Multiple Layers of Neural Networks (logic that emulates neurons in the brain) and complex algorithms train models.

(Interprets complex patterns using filters)

AI Learning Breakdown



Output Configuration

Every Large Language Model (LLM) is different, and to use the most efficient prompting, you need to change not only what you ask, but the settings of the model. YOU can affect how it responds in real time!

Output Length

of tokens (words) the AI will generate in a response. Keep in mind that limiting output length too much will just cut off the output, not make the response shorter.

5 tokens

The more tokens, the more cost, energy, and time needed

50 tokens

Sampling Controls

LLMs do not predict a single token but instead predict the probability of what the next token is based on the input and its vocabulary.

Ex. You type, "The cow says..."

The AI responds "Moo" because that word of its vocabulary related to cows had the greatest probability.

Sampling Controls

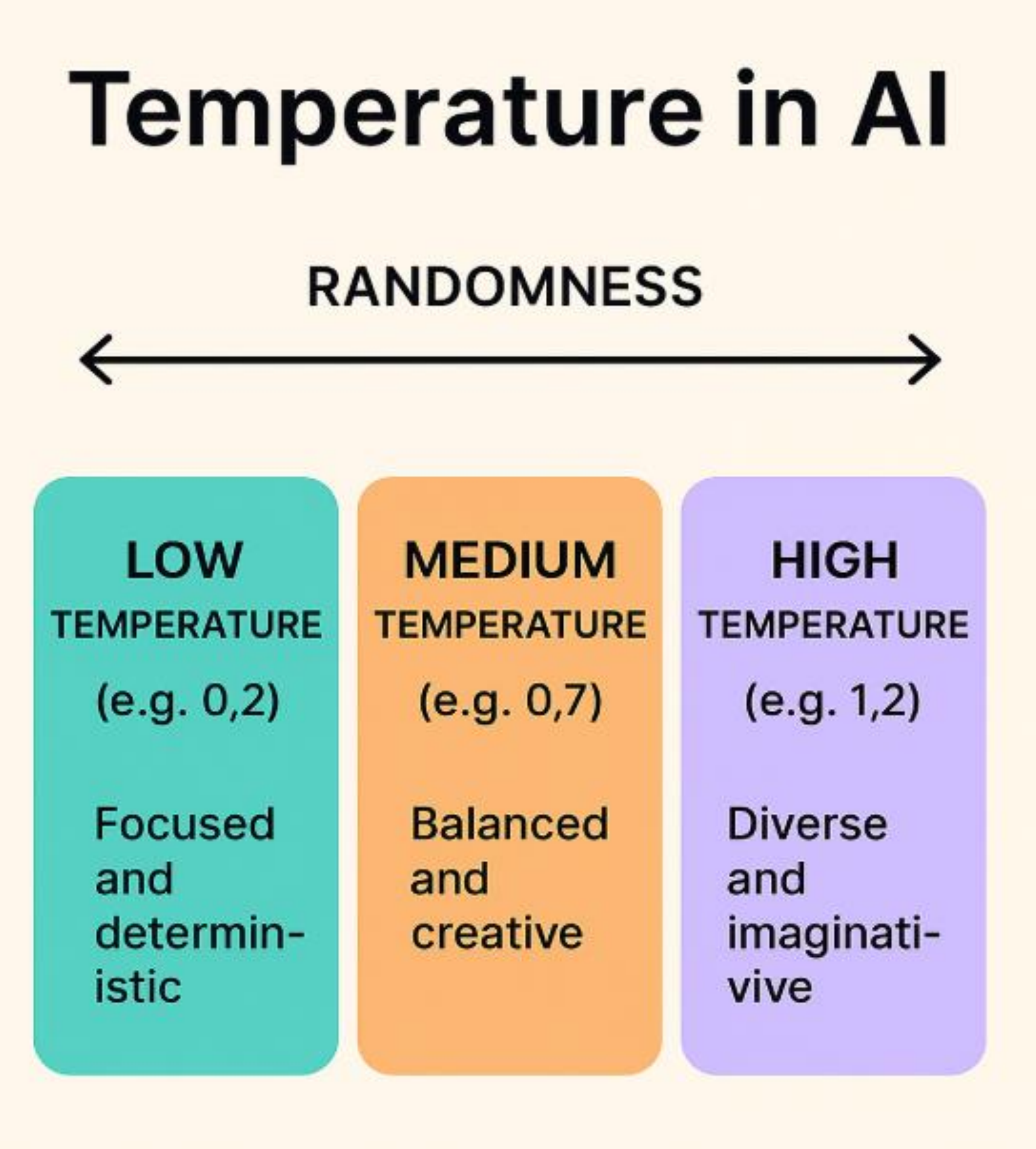
Temperature

Temperature determines the randomness of the token. The higher the temperature, the more creative the prompt.

Top-K and Top-P

Top-K functions similarly to temperature, where the higher it is, the more creative the output.

Top-P can be used to limit the token probabilities of a token getting picked.



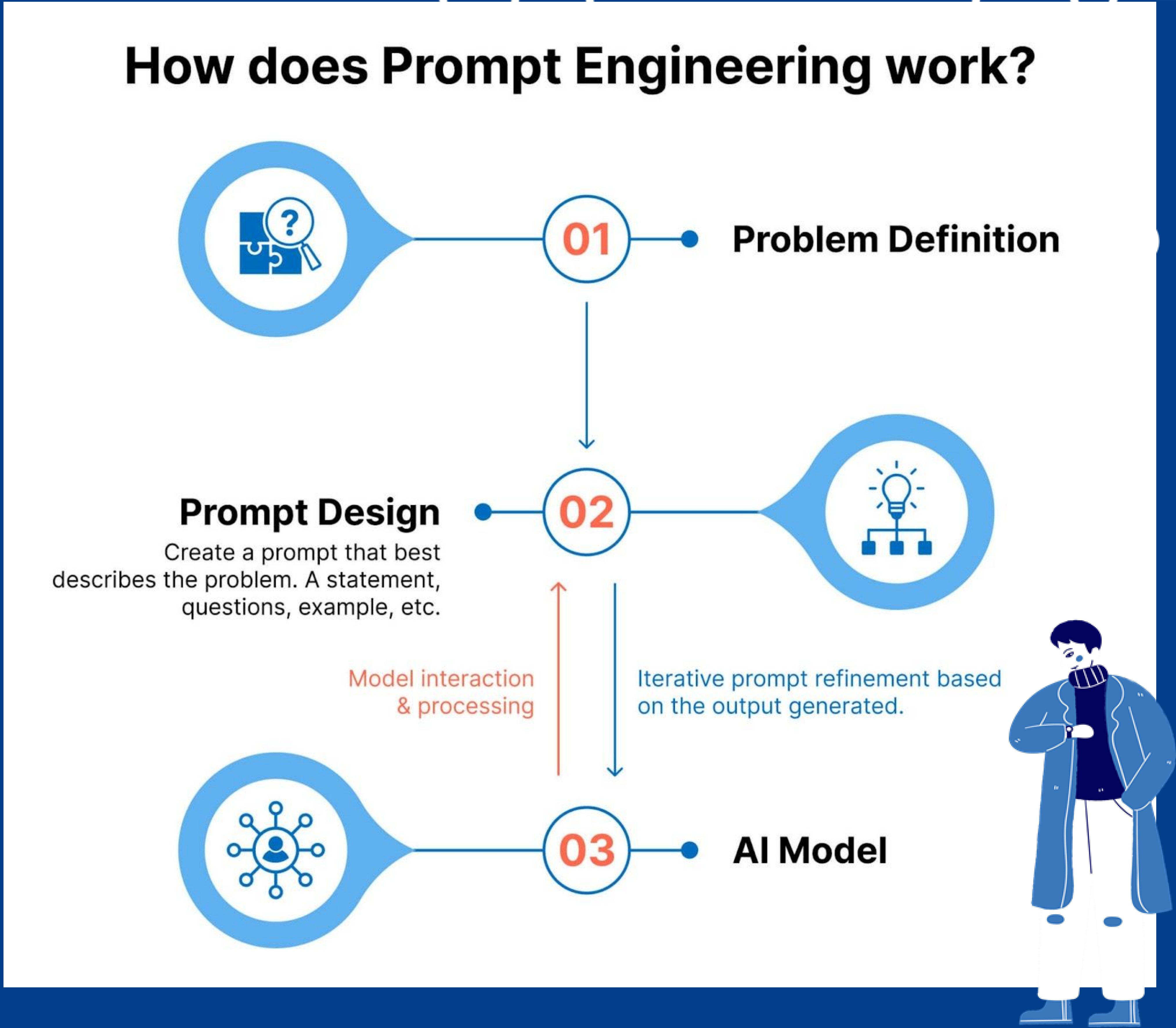
All in all, the more freedom you give a model, the less relevant it may be in your answers.

How Do We Use It?

Prompt Engineering

AI models use their training to predict what kind of output you need. It provides a token output (about 4 characters) and presents the next token based on the earlier tokens and training it went through.

The more specific the prompt, the better the output!



Gemini

OpenAI | Microsoft

Claude AI Pro

perplexity

Google AI Studio

While these strategies can help you make a good prompt, you can also use them to curate your own model, like we did. Google AI Studio provides access to SLMs and AI prototyping that can use prompt engineering strategies to form a model that fits your needs.

Prompt Engineering

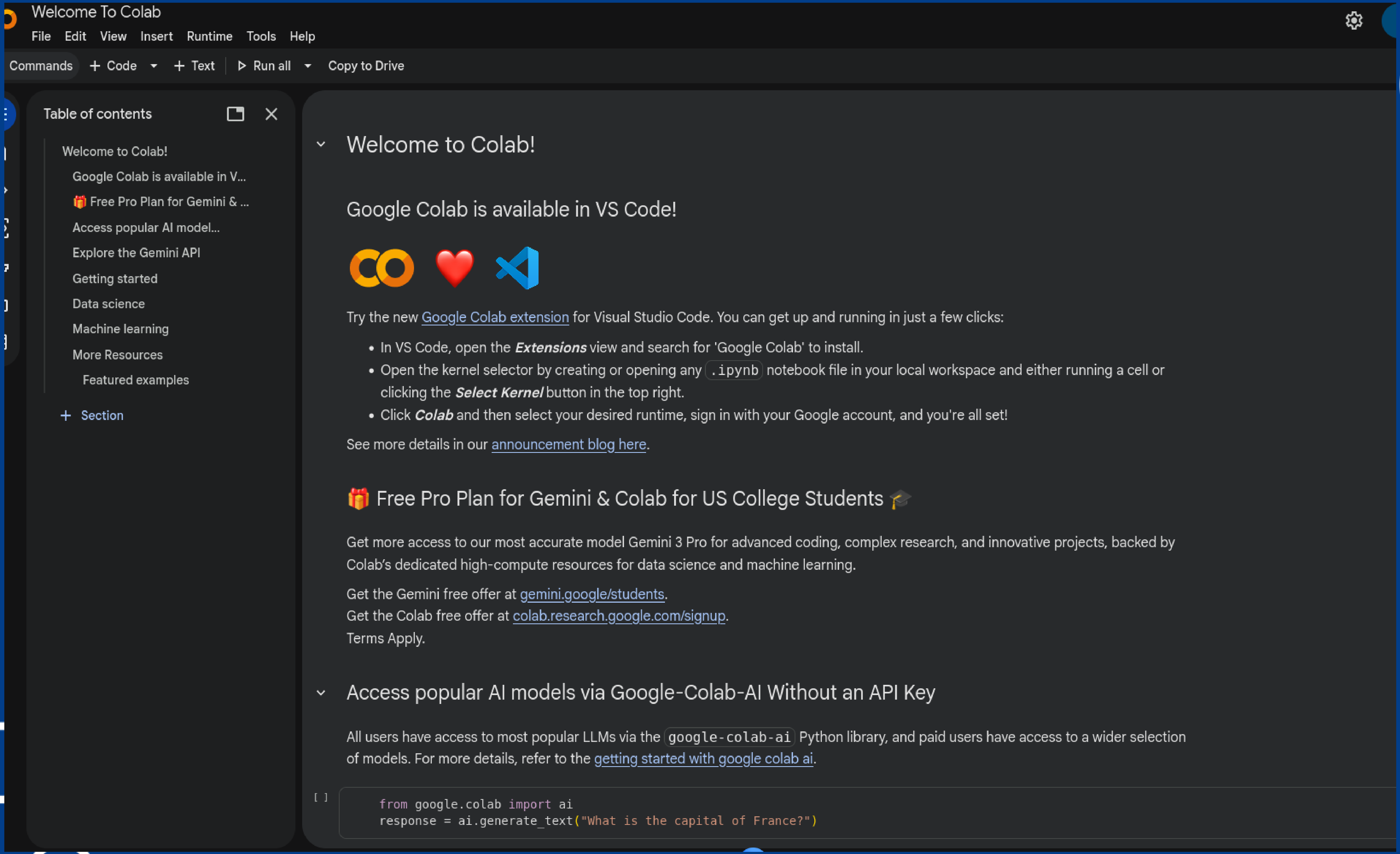
Role Prompting

A technique where you explicitly instruct an AI to assume a specific role when generating responses. This will change how the AI interacts with the user with a distinctive style, tone, and relevant content that the AI is curated towards.

An example of our own code!

```
SYSTEM_PROMPT = (
    "You are Pi, a warm and encouraging tech helper for kids and beginners. "
    "Your job is to make technology feel easy, fun, and stress-free.\n\n"
    "How you communicate:\n"
    "- Keep responses SHORT. 2 to 4 sentences max. If more is needed, break it into a follow-up.\n"
    "- Use simple, everyday words. If you must use a tech term, explain it in one short phrase right away.\n"
    "- Use fun real-world comparisons to explain things (e.g., 'A variable is like a labeled box').\n"
    "- Sound like a friendly, patient older sibling — warm, casual, and never overwhelming.\n"
    "- Never make anyone feel bad for not knowing something.\n\n"
```

Open Google Collab



Download the file

Build_Your_Own_AI_agent_Workshop.ipynb

Upload the file

File -> Upload notebook -> upload > Browse

Run the first block

1. Downloading Libraries

[2] 5s

```
!pip install google-genai
```

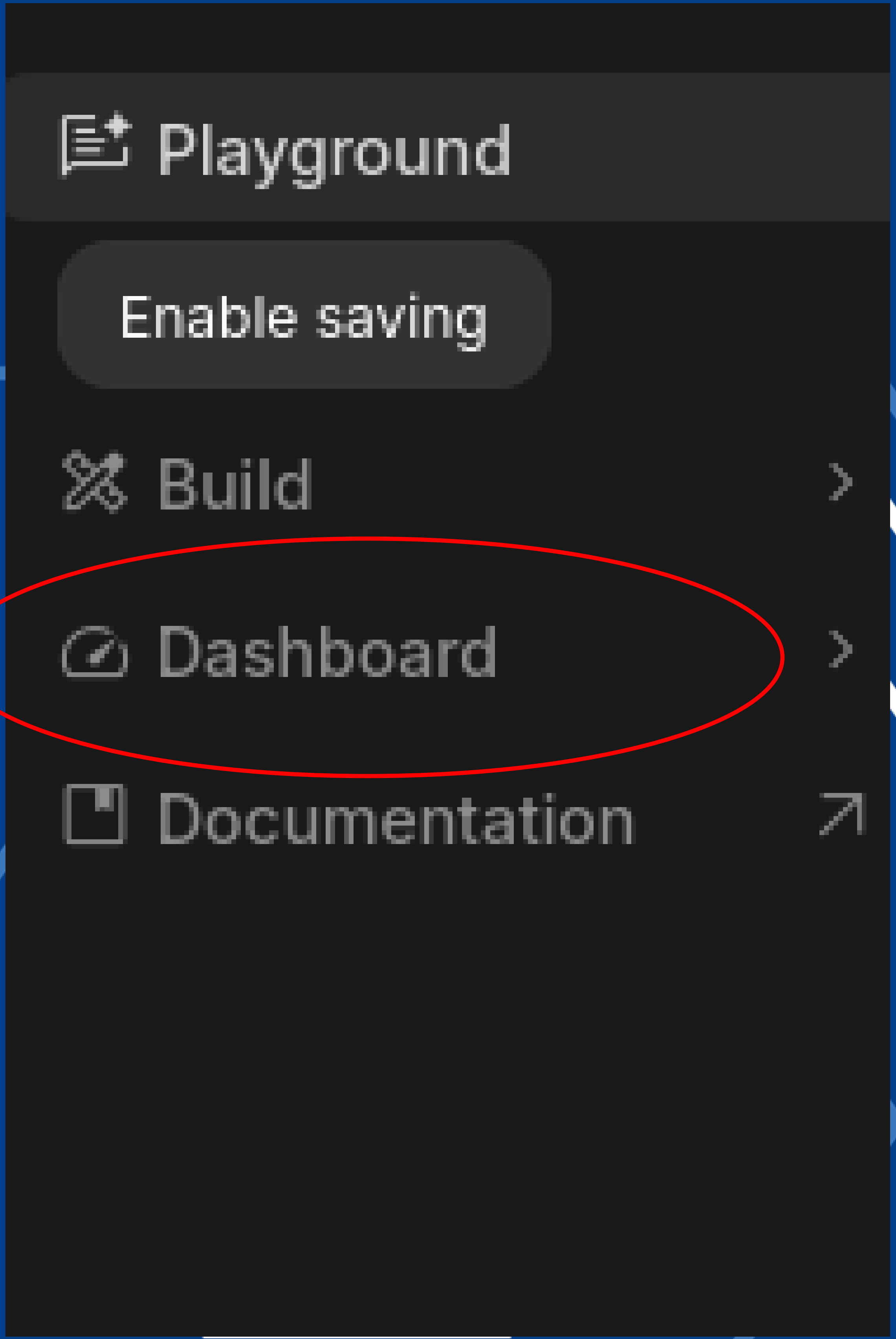
```
Requirement already satisfied: google-genai in /usr/local/lib/python3.12/dist-packages (1.67.0)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (4.12.1)
Requirement already satisfied: google-auth<3.0.0,>=2.47.0 in /usr/local/lib/python3.12/dist-packages (from google-auth[requests]<3.0.0,>=2.47.0->google-genai) (2.47.0)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai) (0.28.1)
Requirement already satisfied: pydantic<3.0.0,>=2.9.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (2.12.3)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai) (2.32.4)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from google-genai) (9.1.4)
Requirement already satisfied: websockets<17.0,>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (15.0.1)
Requirement already satisfied: typing-extensions<5.0.0,>=4.11.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (4.15.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.9.0)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0->google-genai) (3.11)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.47.0->google-auth[requests]<3.0.0,>=2.47.0->google-genai) (0.4.2)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.47.0->google-auth[requests]<3.0.0,>=2.47.0->google-genai) (4.9.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai) (2026.2.25)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0,>=0.28.1->google-genai) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.9.0->google-genai) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.9.0->google-genai) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.9.0->google-genai) (0.4.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai) (3.4.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai) (2.5.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.47.0->google-auth[requests]<3.0.0,>=2.47.0->google
```

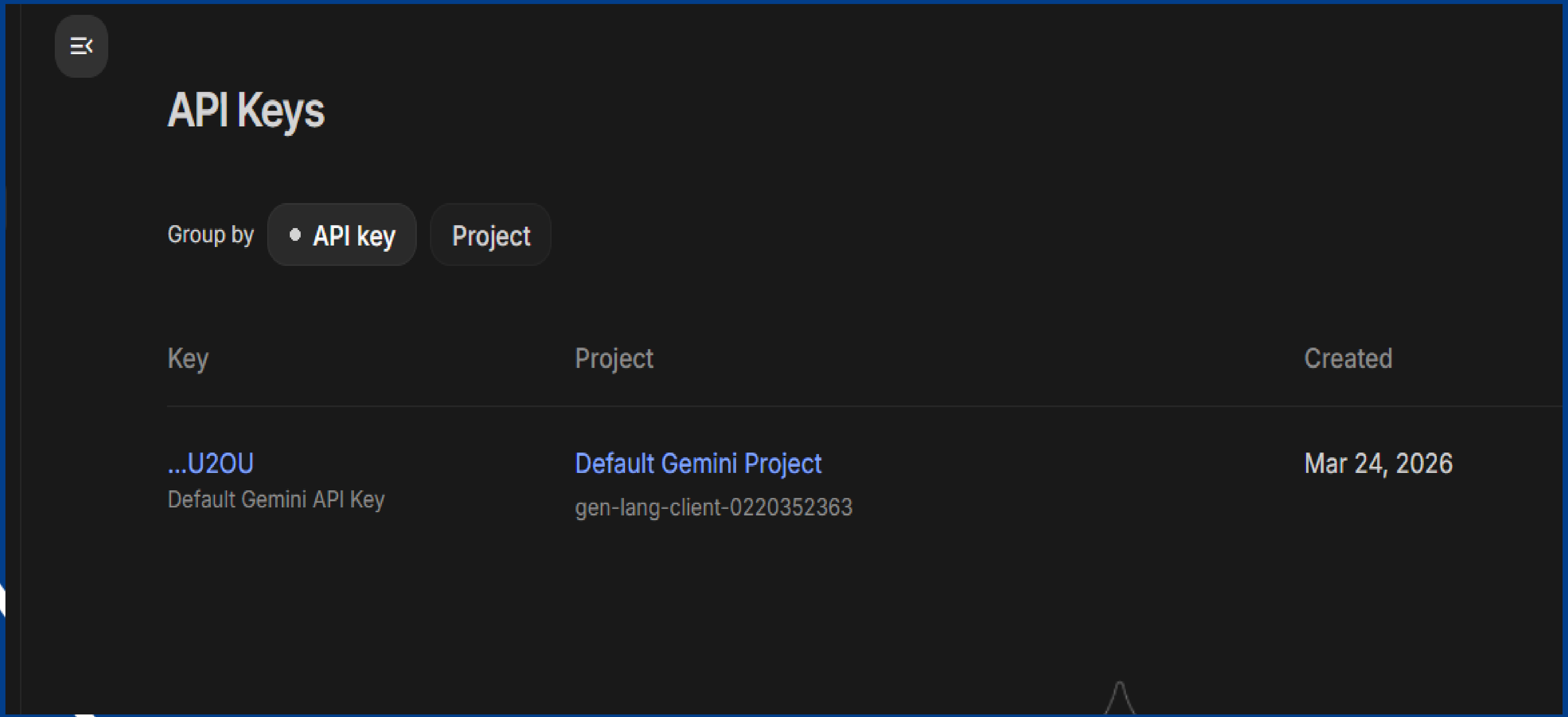
Time to make your own API Key!

STEP 1:Setting up Google Ai Studio

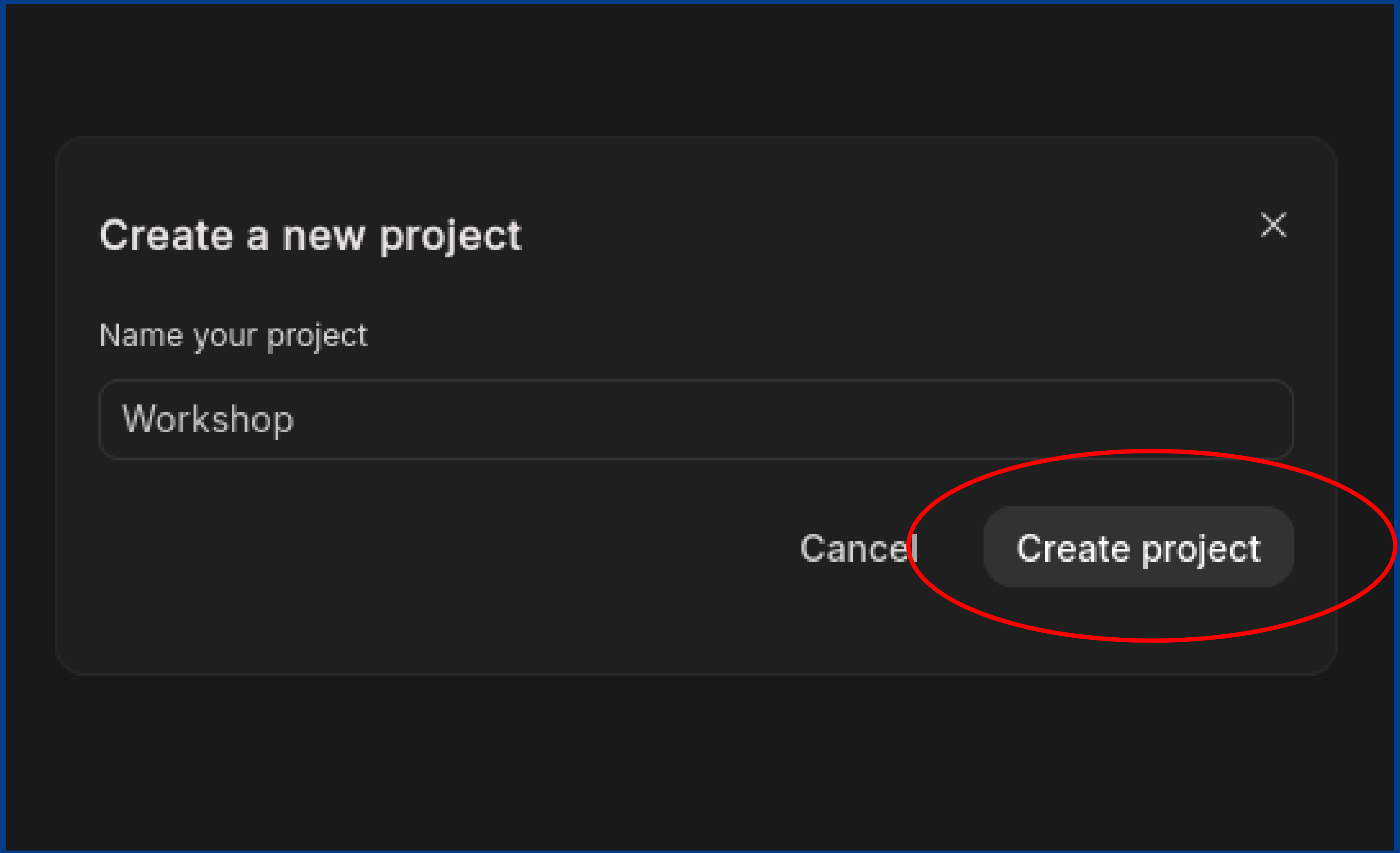
https://aistudio.google.com/prompts/new_chat

STEP 2: Go To Dashboard

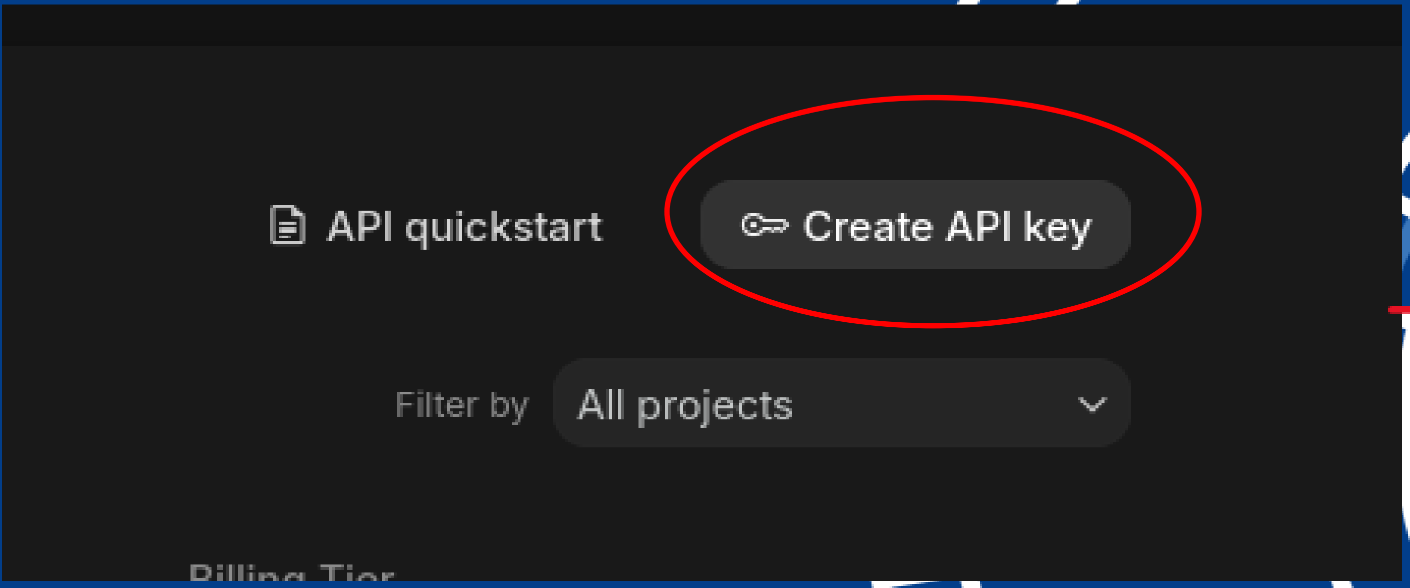


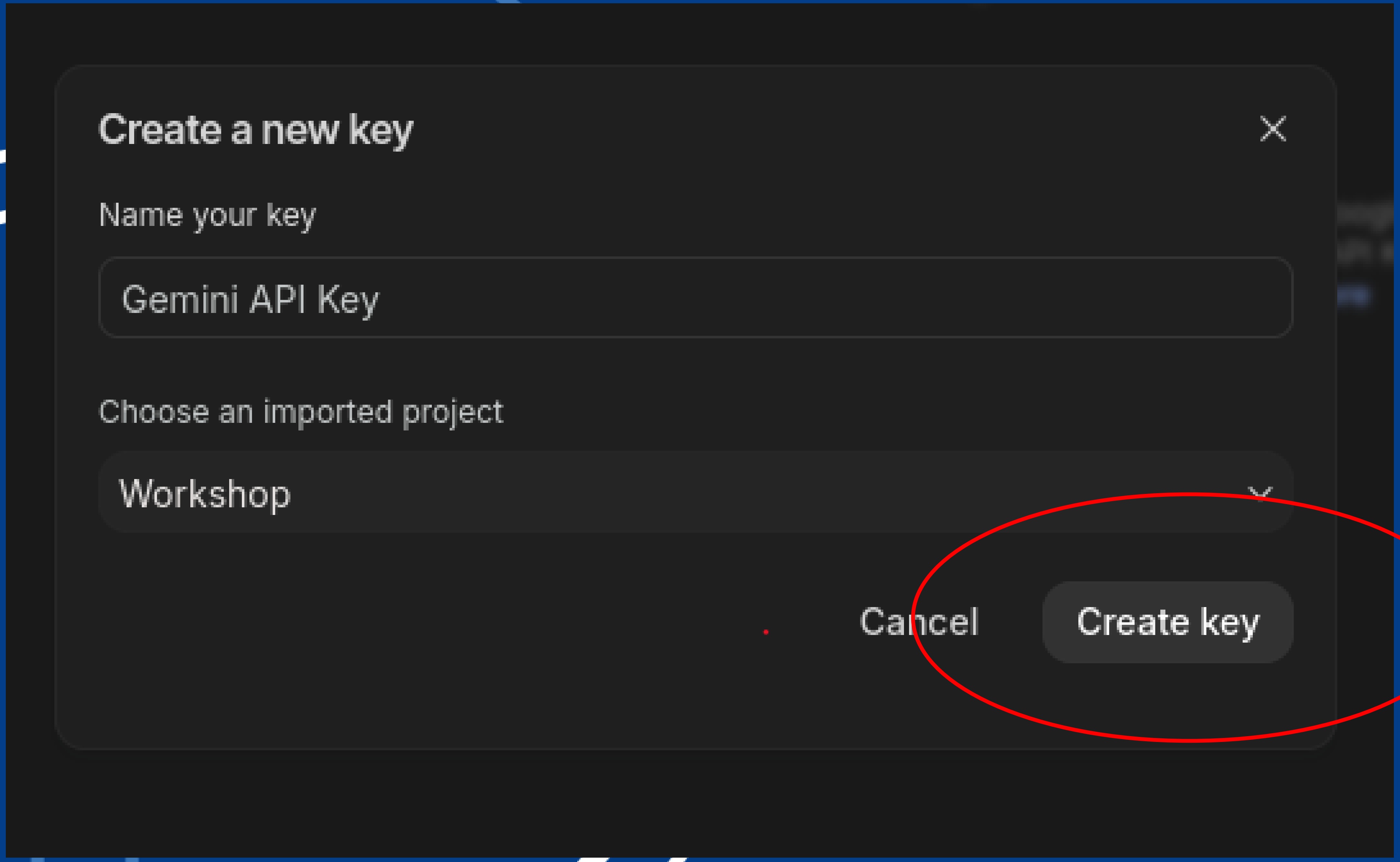


STEP 3: Make Project

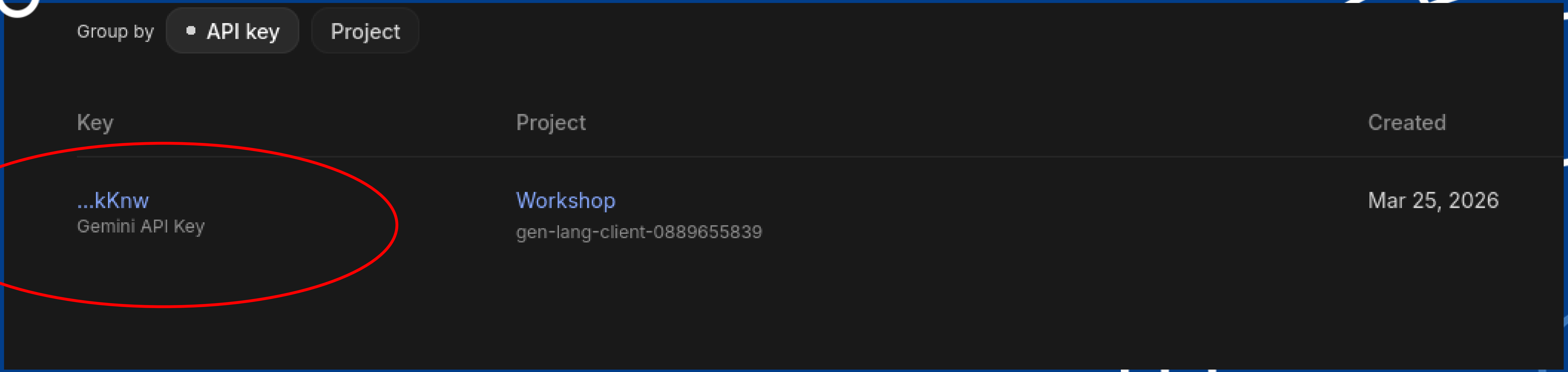


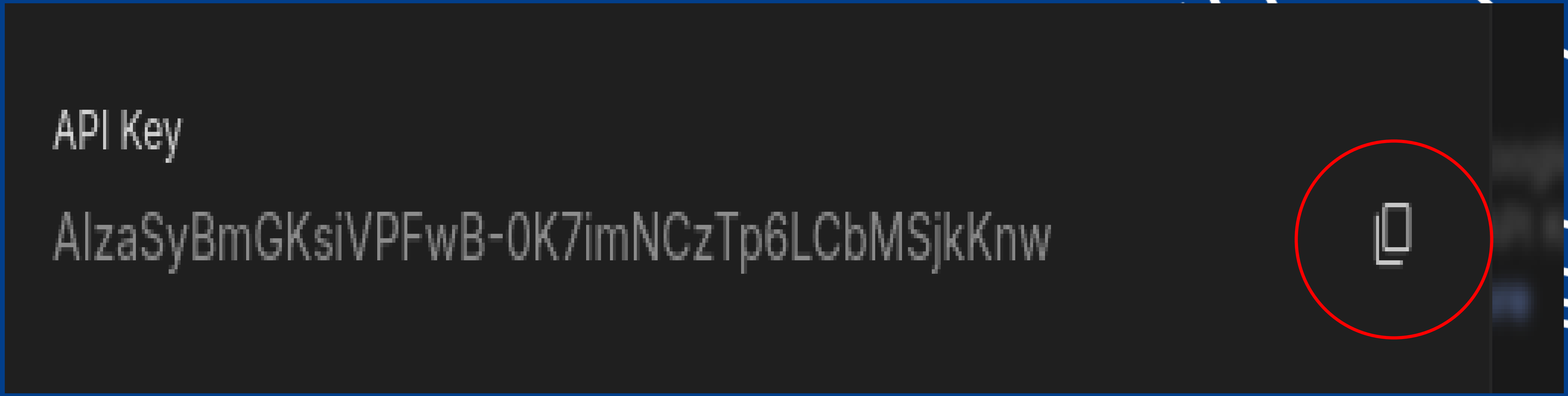
STEP 4: Create The Key





STEP 5:Copy Key to collab





STEP 5: Cont.

2. Setting up Api key

```
import json
import re
from google import genai

# Paste your API key from Google AI Studio here
API_KEY = "YOUR_API_KEY_HERE"
```

STEP 6: The Fun Part

3. Build your own agent

```
  # --- NAME ---
  # What is your agent called?
agent_name = "INSERT NAME HERE"

  # --- BEHAVIOR ---
  # How should your agent act and communicate?
  # Example: "Be friendly and encouraging" / "Be funny and sarcastic" / "Be calm and professional"
agent_behavior = "INSERT BEHAVIOR HERE"

  # --- RULES ---
  # What must your agent always or never do?
  # Example: "Always end with a fun fact" / "Never give direct answers, only hints"
rule_1 = "INSERT RULE HERE"
rule_2 = "INSERT RULE HERE"
rule_3 = "INSERT RULE HERE"



SYSTEM_PROMPT = f"""
  You are {agent_name}.

  Behavior:
  {agent_behavior}

  Rules you must follow:
  1. {rule_1}
  2. {rule_2}
  3. {rule_3}
  """

print(f"Agent '{agent_name}' is ready!")
```

STEP 6: Example

```
  # --- NAME ---
  # What is your agent called?
agent_name = "Song Agent"

  # --- BEHAVIOR ---
  # How should your agent act and communicate?
  # Example: "Be friendly and encouraging" / "Be funny and sarcastic" / "Be calm and professional"
agent_behavior = "You are going to give me rock songs based off my mood"

  # --- RULES ---
  # What must your agent always or never do?
  # Example: "Always end with a fun fact" / "Never give direct answers, only hints"
rule_1 = "only give me 5 songs"
rule_2 = "must be rock songs only"
rule_3 = "Only recommend me songs on spotify"



SYSTEM_PROMPT = f"""
  You are {agent_name}.

  Behavior:
  {agent_behavior}

  Rules you must follow:
  1. {rule_1}
  2. {rule_2}
  3. {rule_3}
  """

print(f"Agent '{agent_name}' is ready!")
```

```
Agent 'Song Agent' is ready!
```

STEP 6:Example cont..

```
# Test your agent!
chat("hello how are you")
```

...

hello how are you

I'm doing great and ready to find some awesome rock for you!

What kind of mood are you in today? Tell me, and I'll whip up some killer tunes.

Song Agent

STEP 6: Example Cont..

5. Keep Chatting

```
chat("i feel happy")
```

hello how are you

You

I'm doing great and ready to find some awesome rock for you!

What kind of mood are you in today? Tell me, and I'll whip up some killer tunes.

Song Agent

i feel happy

You

Awesome! Nothing beats a good rock anthem when you're feeling happy. Here are 5 songs on Spotify to keep that energy high:

1. **"Don't Stop Me Now"** by Queen
2. **"Mr. Brightside"** by The Killers
3. **"Livin' on a Prayer"** by Bon Jovi
4. **"Are You Gonna Be My Girl"** by Jet
5. **"All My Life"** by Foo Fighters

Crank 'em up and enjoy!

Song Agent

RESULTS - Survey

Thank you!

Any Questions?